initCKF: Initializing the CKF

- In this lesson, you will learn how to implement the CKF in Octave, and will apply it to the nonlinear spring-mass-damper.
- The CKF initialization code is similar to its SPKF counterpart.

```
function ckfData = initCKF(xhat0,SigmaX0,priorU,model)
  ckfData.xhat   = xhat0;          ckfData.SigmaX = SigmaX0;      % Copy some
  ckfData.SigmaV = model.SigmaV;   ckfData.SigmaW = model.SigmaW; % parameters
  ckfData.priorU = priorU;         ckfData.Qbump  = 5;            % into ckfData
  ckfData.model  = model;          ckfData.Sw     = model.Sw;

  % CKF specific parameters
  nx = length(ckfData.xhat);          ckfData.nx = nx; % state-vector length
  ckfData.nz = 1;                     ckfData.nu = 1;  % meas, input-vector length

  ckfData.h = sqrt(nx);  % CKF tuning factor
  W1 = 0; W2 = 1/(2*nx); % weighting factors when computing mean and covar
  ckfData.Wm = [W1; W2*ones(2*nx,1)]; % mean weighting factors
  ckfData.Wc = ckfData.Wm;            % covariance weighting factors
```


iterCKF: Unpacking parameters

- The iterCKF function has important differences from the iterSPKF function, so we will look at all of it.

```
function [xhat,bounds,ckfData] = iterCKF(uk,zk,dT,ckfData)
  model = ckfData.model;
  SQRT = @(x) sqrt(max(0,x)); % "Safe" square root

  % Get data stored in spkfData structure
  priorU = ckfData.priorU;
  SigmaX = ckfData.SigmaX;
  xhat = ckfData.xhat;
  nx = ckfData.nx;
  Wc = ckfData.Wc;
```


iterCKF: Step 1a

- The Octave code next implements Step 1a.

```
  % Step 1a: State estimate time update
  % Step 1a-1: Create cholesky decomposition of SigmaX
  [sigmaX,p] = chol(SigmaX,'lower');
  if p>0
    fprintf('Cholesky errorin step 1a.  Recovering...\n');
    sigmaX = SQRT(diag(SigmaX));
  end % NOTE: sigmaX is lower-triangular

  % Step 1a-2: Calculate SigmaX points (strange indexing of xhata to
  % avoid "repmat" call, which is very inefficient in MATLAB)
  X = xhat(:,ones([1 2*nx+1])) + ...
       ckfData.h*[zeros([nx 1]), sigmaX, -sigmaX];

  % Step 1a-3: Time update from last iteration until now
  %     stateEqn(xold,current,dT)
  Xx = stateEqn(X,priorU,dT);
  xhat = Xx*ckfData.Wm;
```

iterCKF: Steps 1b–1c

- The Octave code next implements Steps 1b–1c.

```
% Step 1b: Error covariance time update
%           - Compute weighted covariance sigmaminus(k)
%             (strange indexing of xhat to avoid "repmat" call)
Xs = Xx - xhat(:,ones([1 2*nx+1]));
SigmaX = Xs*diag(Wc)*Xs';
SigmaX = SigmaX + ckfData.Sw*dT/model.m^2*[0 0; 0 1]; % See Lesson 3.1.5
% Alternate...
% SigmaX = SigmaX + ckfData.SigmaW;

% Step 1c: Output estimate
%           - Compute weighted output estimate zhat(k)
[sigmaX,p] = chol(SigmaX,'lower');
if p>0
  fprintf('Cholesky error in step 1c.  Recovering...\n');
  sigmaX = SQRT(diag(SigmaX));
end
X = xhat(:,ones([1 2*nx+1])) + ckfData.h*[zeros([nx 1]), sigmaX, -sigmaX];
Z = outputEqn(X,model);
zhat = Z*ckfData.Wm;
```


iterCKF: Steps 2a–2c

- The Octave code next implements Steps 2a–2c.

```
% Step 2a: Estimator gain matrix
Zs = Z - zhat(:,ones([1 2*nx+1]));
SigmaXZ = Xs*diag(Wc)*Zs';
SigmaZ  = Zs*diag(Wc)*Zs' + ckfData.SigmaV;
L = SigmaXZ/SigmaZ;

% Step 2b: State estimate measurement update
r = zk - zhat; % innovation: use to check for sensor errors...
if r^2 > 100*SigmaZ, L(:)=0.0; end
xhat = xhat + L*r;

% Step 2c: Error covariance measurement update
SigmaX = SigmaX - L*SigmaZ*L';
```


iterCKF: Cleanup

- The Octave code next makes sure that SigmaX makes sense, and saves data for the next iteration.

```
% Q-bump code
if r^2 > 16*SigmaZ % bad output estimate by 4 std. devs, bump Q
  fprintf('Bumping SigmaX\n');
  SigmaX = SigmaX*ckfData.Qbump;
end
[~,S,V] = svd(SigmaX); % Implement Higham method to help robustness
HH = V*S*V';
SigmaX = (SigmaX + SigmaX' + HH + HH')/4;

% Save data in ckfData structure for next time...
ckfData.priorU = uk;
ckfData.SigmaX = SigmaX;
ckfData.xhat = xhat;
bounds = 3*sqrt(diag(SigmaX));
```

`iterCKF`: State and output equations

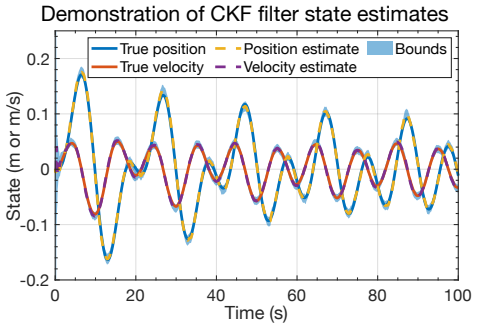
- The state and output equations for this model are implemented in two nested functions inside the `iterCKF` function:

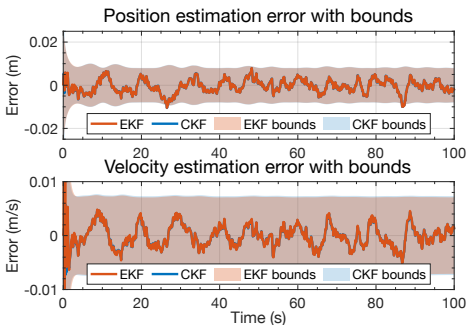
```
  % Calculate new states for all of the old state vectors in xold.
  function xnew = stateEqn(xold,priorU,dT)
    % Compute linear homogeneous part
    xnew = [1 dT; -model.k1*dT/model.m 1-model.b*dT/model.m]*xold;
    % Add on nonlinear and forcing part
    xnew(2,:) = xnew(2,:) - model.k2*dT*xold(1,:).^3/model.m + ...
                dT/model.m*priorU;
  end

  % Calculate cell output voltage for all of state vectors in xhat
  function zhat = outputEqn(xhat,model)
    zhat = atan(xhat(1,:)/model.d);
  end
end
```


Example EKF/CKF results

- Some sample results are shown below, comparing EKF and CKF performance on the spring-mass-damper dataset.
  - RMSE is 0.0027433 versus (0.0027576 for EKF).
  - Percent of time error outside bounds = 0.3996 % (versus 0.4496 % for EKF).





Summary

- You have learned how to implement the CKF in Octave code.
- It is very similar to the generic SPKF, but:
  - It does not require creating augmented state and covariance matrices,
  - And it does require implementing two Cholesky decompositions instead of only one.
- Results for the spring-mass-damper dataset are arguably better than for EKF and even for generic SPKF.
  - This will not always be true—I recommend trying different flavors of nonlinear KF for your specific application.